

Testes Automatizados no Projeto da Turma

Documentação da atividade – Aponti Academy 2026 – Turma 5 de DevOps

1. Integrantes do Grupo e Responsabilidades

A tabela abaixo apresenta os integrantes do grupo e a divisão de responsabilidades adotada, alinhada às partes da atividade proposta.

Integrante	Função	Responsabilidades
Otávio Sousa Leão de Barros	Líder Técnico e Infraestrutura (Git/GitHub)	Fork do projeto e adição dos colaboradores, inicialização do projeto Node.js (criação do package.json), configuração do script de testes nativo, criação da estrutura de pastas de testes e abertura do Pull Request final para o repositório principal.
Luiz Felipe Gomes Diogo e Mariana Araújo de Sousa	Analistas de QA (Regras de Negócio)	Análise do projeto e identificação dos comportamentos esperados (Parte 1); preenchimento da tabela de casos de teste com Situações, Entrada e Resultado Esperado (Parte 2).
Lucas Henrique Dias de Medeiros e Leonardo Antônio da Silva	Engenheiros de Automação de Testes	Implementação dos testes em JavaScript/Node.js seguindo a lógica Entrada → Processamento → Resultado Esperado (Parte 3).
Nayara Karla Medeiros	Especialista em Estratégia de Testes	Criação do Smoke Test, do Teste de Sanidade e da Suíte de Regressão; execução local dos testes e registro dos resultados (Partes 4 e 5).
Vitor Marques de Freitas	Tech Writer e Coordenador de Apresentação	Consolidação da documentação completa da atividade e criação dos slides de apresentação (10 minutos).

2. Objetivo

Criar testes automatizados para validar o funcionamento do projeto colaborativo da turma e prepará-los para futura execução em uma pipeline de CI/CD. O projeto avaliado é o repositório da Turma 5 de DevOps da Aponti Academy, especificamente o script `scripts/gerar-readme.js`, responsável por ler os arquivos JSON dos alunos, validar seus dados, remover duplicidades, ordenar os registros e gerar a tabela de integrantes no `README.md`.

3. Parte 1 – Identificação dos Comportamentos Esperados

A partir da leitura do código-fonte de `scripts/gerar-readme.js`, foram identificadas cinco funções principais e seus respectivos comportamentos esperados:

Aqui está o conteúdo organizado em uma lista, com as duplicatas do texto original removidas para facilitar a leitura:

- **T01 | Situação:** Pasta alunos existe | **Entrada:** Pasta contendo arquivos | **Resultado esperado:** O script executa normalmente.
- **T02 | Situação:** Pasta alunos não existe | **Entrada:** Pasta ausente | **Resultado esperado:** O script exibe a mensagem "Pasta 'alunos' não encontrada." e encerra a execução.
- **T03 | Situação:** Arquivo JSON válido | **Entrada:** JSON contendo nome e github | **Resultado esperado:** O aluno é incluído na lista e aparece na tabela do README.
- **T04 | Situação:** Arquivo sem nome | **Entrada:** JSON sem a propriedade nome | **Resultado esperado:** O registro é ignorado e não aparece na tabela.
- **T05 | Situação:** Arquivo sem github | **Entrada:** JSON sem a propriedade github | **Resultado esperado:** O registro é ignorado e não aparece na tabela.
- **T06 | Situação:** Arquivo com extensão diferente de .json | **Entrada:** .txt, .png, .pdf | **Resultado esperado:** O arquivo é ignorado.
- **T07 | Situação:** JSON inválido | **Entrada:** Arquivo JSON mal formatado | **Resultado esperado:** O erro é tratado e a execução continua normalmente.
- **T08 | Situação:** GitHub duplicado | **Entrada:** Dois arquivos com o mesmo usuário GitHub | **Resultado esperado:** Apenas um aluno é mantido na lista final.
- **T09 | Situação:** Alunos fora de ordem | **Entrada:** Arquivos em ordem aleatória | **Resultado esperado:** A tabela é gerada em ordem alfabética pelo nome.

- **T10 | Situação:** LinkedIn ausente | **Entrada:** JSON sem linkedin | **Resultado esperado:** A coluna LinkedIn exibe "-".
- **T11 | Situação:** Cidade ausente | **Entrada:** JSON sem cidade | **Resultado esperado:** A coluna Cidade exibe "-".
- **T12 | Situação:** Atualização do README | **Entrada:** Execução do script | **Resultado esperado:** O README é atualizado com a nova tabela e estatísticas.
- **T13 | Situação:** Estatísticas | **Entrada:** Execução do script | **Resultado esperado:** O total de alunos cadastrados corresponde ao número de alunos válidos.

4. Parte 2 – Casos de Teste

Com base na análise da Parte 1, foram definidos os seguintes casos de teste:

ID	Situação	Entrada	Resultado Esperado
T01	Pasta alunos existente	Pasta contendo arquivos JSON válidos	O script é executado normalmente e inicia o processamento dos arquivos.
T02	Pasta alunos inexistente	Pasta alunos removida ou inexistente	O sistema exibe a mensagem "Pasta 'alunos' não encontrada." e encerra a execução.
T03	JSON válido	Arquivo .json contendo nome e github válidos	O aluno é incluído na lista e sua linha é adicionada à tabela do README.
T04	Nome ausente	JSON sem a propriedade nome	O registro é ignorado e não aparece na tabela do README.
T05	GitHub ausente	JSON sem a propriedade github	O registro é ignorado e não aparece na tabela do README.
T06	Arquivo que não é JSON	Arquivos .txt, .png ou .pdf na pasta alunos	O arquivo é ignorado e não é processado.
T07	JSON inválido	Arquivo .json com erro de sintaxe	O erro é tratado pelo catch, uma mensagem é exibida e a execução continua.

T08	GitHub duplicado	Dois arquivos com o mesmo usuário GitHub	Apenas um registro é mantido na lista final e exibido no README.
T09	Alunos fora de ordem	Arquivos contendo nomes em ordem aleatória	Os alunos são ordenados alfabeticamente pelo campo nome antes da geração da tabela.
T10	LinkedIn ausente	JSON sem a propriedade linkedin	A coluna LinkedIn exibe "-" na tabela do README.
T11	Cidade ausente	JSON sem a propriedade cidade	A coluna Cidade exibe "-" na tabela do README.
T12	Atualização do README	Execução completa do script	O README é atualizado com a tabela de alunos, total de cadastrados e data da última atualização.
T13	Atualização das estatísticas	Execução do script com alunos válidos	O total de alunos cadastrados corresponde ao número de registros válidos processados.

5. Parte 3 – Implementação dos Testes

Os testes foram estruturados para o ecossistema **JavaScript/Node.js**, utilizando o **Node.js Test Runner (node:test)**, ferramenta nativa do Node.js compatível com a criação e execução de testes automatizados e adequada para integração em futuras pipelines de CI/CD.

Estrutura de pastas utilizada:

```
projeto/
├── alunos/
├── scripts/
│   └── gerar-readme.js
├── tests/
│   └── gerar-readme.test.js
├── README.md
└── package.json
```

Cada teste implementado segue a lógica **ENTRADA → PROCESSAMENTO → RESULTADO ESPERADO**. Inicialmente são preparados os dados de entrada (como arquivos JSON válidos, inválidos ou registros simulados). Em seguida, a função correspondente do script é executada. Por fim, o resultado obtido é comparado com o resultado esperado por meio das

asserções da biblioteca **node:assert**, verificando se o comportamento da aplicação está de acordo com os requisitos definidos.

6. Parte 4 – Classificação dos Testes

Smoke Test – O sistema está funcionando?

Teste utilizado: T01 – Pasta alunos existente

O que foi validado:

Verifica se o sistema consegue acessar a pasta **alunos** e executar a leitura dos arquivos sem lançar erros, garantindo que a funcionalidade principal do projeto está operacional.

Resultado:  Aprovado

Justificativa:

Este teste foi classificado como **Smoke Test** porque verifica rapidamente se a aplicação está minimamente operacional. Caso a pasta de alunos não pudesse ser acessada ou o script apresentasse erro logo no início, não seria possível executar os demais testes.

Teste de Sanidade – A alteração específica funcionou?

Teste utilizado: T09 – Ordenação alfabética pelo campo nome

O que foi validado:

Verifica se os alunos são ordenados corretamente em ordem alfabética antes da geração da tabela do README.

Resultado:  Aprovado

Justificativa:

Este teste foi classificado como **Teste de Sanidade** porque valida uma funcionalidade específica do sistema, confirmando que a ordenação continua funcionando corretamente após alterações no código.

Teste de Regressão – O que já funcionava continua funcionando?

Foram utilizados os seguintes testes:

- **T02** – Pasta alunos inexistente.
- **T03, T10 e T11** – Geração da tabela e tratamento de propriedades opcionais ausentes.

- **T04, T05, T06 e T07** – Validação da leitura de arquivos (JSON inválido, arquivos que não são JSON e registros incompletos).
- **T08** – Remoção de usuários do GitHub duplicados.
- **T12** – Estrutura da tabela gerada no README.
- **T13** – Fluxo completo de processamento e estatísticas do projeto.

Justificativa:

Esses testes foram classificados como **Testes de Regressão** porque verificam funcionalidades já implementadas no sistema, garantindo que alterações realizadas no código não comprometam comportamentos existentes, como validação dos arquivos, tratamento de erros, remoção de duplicidades, geração da tabela e atualização das estatísticas.

7. Parte 5 – Suíte de Testes e Resultados da Execução

Foi desenvolvida uma suíte mínima de testes para validar o funcionamento do projeto, contemplando os três principais tipos de testes estudados: **Smoke Test**, **Teste de Sanidade** e **Testes de Regressão**. Os testes foram executados localmente utilizando o Node.js Test Runner, garantindo a verificação das principais funcionalidades do sistema.

Teste	Tipo	O que valida?	Resultado
T01	Smoke	Verifica se a pasta alunos pode ser acessada e se a leitura dos arquivos ocorre sem erros	✓ Aprovado
T09	Sanidade	Verifica se os alunos são ordenados corretamente em ordem alfabética antes da geração da tabela do README.	✓ Aprovado
T02	Regressão	Valida o tratamento da ausência da pasta alunos , verificando se o erro é tratado corretamente.	✓ Aprovado
T04, T05, T06 e T07	Regressão	Verifica o tratamento de arquivos inválidos, registros incompletos, arquivos que não são JSON e JSON com erro de sintaxe.	✓ Aprovado
T08	Regressão	Verifica se usuários do GitHub duplicados são removidos corretamente, independentemente do uso de letras maiúsculas ou minúsculas	✓ Aprovado

T03, T10 e T11	Regressão	Valida a geração da tabela do README e o tratamento correto de propriedades opcionais ausentes	✓ Aprovado
T12	Regressão	Verifica se a estrutura da tabela gerada no README segue o formato esperado	✓ Aprovado
T13	Regressão	Valida o fluxo completo de leitura, remoção de duplicidades, ordenação e geração das estatísticas do projeto	✓ Aprovado

Execução dos testes

Os testes foram executados localmente utilizando o comando:

```
node --test
```

Durante a execução foi exibida uma mensagem informando um erro ao ler um arquivo JSON inválido. Esse comportamento era esperado e fazia parte do teste **T07**, cujo objetivo era verificar se o sistema tratava corretamente arquivos com erro de sintaxe sem interromper a execução dos demais testes.

Ao final da execução foram obtidos os seguintes resultados:

- **Total de testes executados:** 8
- **Testes aprovados:** 8
- **Testes reprovados:** 0
- **Tempo aproximado de execução:** 366 ms

Os resultados obtidos demonstram que todas as funcionalidades avaliadas atenderam aos comportamentos esperados. Os oito testes executados foram aprovados com sucesso, evidenciando que o sistema está operacional, realiza corretamente a validação dos arquivos, remove registros duplicados, ordena os alunos adequadamente e gera o README conforme os requisitos definidos.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\DELL\Desktop\testes automatizados - grupo 2\aponti-fap-devops-turma5-2026> node --test
Erro ao ler invalido.json: Expected property name or '}' in JSON at position 2 (line 1 column 3)
✓ T01 - Pasta alunos existente (42.9609ms)
✓ T02 - Pasta alunos inexistente (3.8547ms)
✓ T04, T05, T06 e T07 - Validação na leitura de arquivos (Faltando dados ou inválidos) (30.3991ms)
✓ T08 - GitHub duplicado deve manter apenas um registro (Case Insensitive) (0.9269ms)
✓ T09 - Ordenação alfabética pelo campo nome (43.9548ms)
✓ T03, T10 e T11 - Geração da Tabela e propriedades opcionais ausentes (0.5792ms)
✓ T12 - Estrutura da Tabela no README (0.5107ms)
✓ T13 - Estatísticas de total de alunos válidos (Integração Local) (22.7143ms)
i tests 8
i suites 0
i pass 8
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 365.9044
PS C:\Users\DELL\Desktop\testes automatizados - grupo 2\aponti-fap-devops-turma5-2026> █
```

Figura 1 – Execução da suíte de testes no terminal do Visual Studio Code